

1. Εισαγωγή στην αντικειμενοστρέφεια

1.1 Κλάση και αντικείμενο

Η κλάση περιγράφει τι πληροφορίες κρατάμε και τι ενέργειες μπορεί να κάνει ένα αντικείμενο. Το αντικείμενο είναι η πραγματική οντότητα που δημιουργείται στη μνήμη όταν τρέχει το πρόγραμμα.

```
class Student {
    String name;
    int semester;
    void printInfo() {
        System.out.println(name + " - " + semester);
    }
}

Student s1 = new Student();
s1.name = "Maria";
s1.semester = 1;
```

Για τους πρωτοετείς το πιο σημαντικό είναι να ξεχωρίσουν ότι η κλάση είναι περιγραφή, ενώ το αντικείμενο είναι αυτό που χρησιμοποιούμε στην εκτέλεση.

2. Βασική δομή προγράμματος Java και εκτέλεση

2.1 Η main

Κάθε απλό πρόγραμμα Java ξεκινά από τη μέθοδο main. Εκεί μπαίνει ο κώδικας που θέλουμε να εκτελεστεί πρώτος.

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java!");
    }
}
```

2.2 Compile και εκτέλεση

Αν το αρχείο λέγεται Hello.java, πρώτα το μεταγλωττίζουμε και μετά το τρέχουμε.

```
javac Hello.java
java Hello
```

2.3 Ορίσματα γραμμής εντολών

Το String[] args περιέχει τις τιμές που γράφουμε μετά το όνομα του προγράμματος όταν το εκτελούμε.

```
public class HelloName {
    public static void main(String[] args) {
        if (args.length > 0) {
            System.out.println("Hello, " + args[0] + "!");
        } else {
            System.err.println("Usage: java HelloName <name>");
        }
    }
}

// Εκτέλεση:
// javac HelloName.java
// java HelloName Kostas
```

2.4 System.out και System.err

Το System.out χρησιμοποιείται για κανονική έξοδο στην οθόνη. Το System.err χρησιμοποιείται για μηνύματα λάθους ή μηνύματα χρήσης όταν κάτι δεν δόθηκε σωστά.

- System.out.println(): κανονικό μήνυμα.
- System.err.println(): μήνυμα σφάλματος ή προειδοποίησης.

3. Μεταβλητές, τύποι και τελεστές

3.1 Μεταβλητές

Μεταβλητή είναι μια θέση στη μνήμη όπου αποθηκεύουμε μια τιμή. Κάθε μεταβλητή έχει τύπο, όνομα και τιμή.

```
int age = 18;
double grade = 8.5;
boolean passed = true;
char letter = 'A';
```

3.2 Πρωταρχικοί τύποι και τύποι αναφοράς

Οι πρωταρχικοί τύποι κρατούν απευθείας τιμές. Οι τύποι αναφοράς κρατούν αναφορά προς αντικείμενο που βρίσκεται στη μνήμη.

- Πρωταρχικοί τύποι: int, double, boolean, char κ.ά.
- Τύποι αναφοράς: String, Student, Scanner, πίνακες, αντικείμενα κλάσεων.
- Οι τύποι αναφοράς μπορούν να έχουν τιμή null.

3.3 Τελεστές

Οι τελεστές χρησιμοποιούνται για πράξεις, συγκρίσεις και λογικούς ελέγχους.

```
int x = 10;
int y = 3;
System.out.println(x + y); // 13
System.out.println(x > y); // true
System.out.println(x != y); // true
boolean ok = (x > 0) && (y > 0);
```

4. String και βασικές μέθοδοι

4.1 Τι είναι String

Το String είναι κλάση που αναπαριστά ακολουθία χαρακτήρων. Το χρησιμοποιούμε για λέξεις, προτάσεις, ονόματα, μηνύματα και γενικά κείμενο.

```
String greeting = "Hello";
String name = "Maria";
String message = greeting + " " + name;
System.out.println(message);
```

4.2 Χρήσιμες μέθοδοι String

Τα String έχουν πολλές έτοιμες μεθόδους για επεξεργασία κειμένου.

```
String s = "Java";
System.out.println(s.length()); // μήκος
System.out.println(s.charAt(0)); // πρώτος χαρακτήρας
System.out.println(s.substring(1,3));
System.out.println(s.toUpperCase());
System.out.println(s.equals("Java"));
```

4.3 Προσοχή στην ισότητα String

Για σύγκριση περιεχομένου σε String χρησιμοποιούμε equals και όχι ==. Το == ελέγχει αν δύο αναφορές δείχνουν στο ίδιο αντικείμενο.

```
String a = "Java";
String b = new String("Java");
System.out.println(a == b); // πιθανό false
System.out.println(a.equals(b)); // true
```

5. Είσοδος από χρήστη και Scanner

5.1 Scanner από πληκτρολόγιο

Ο Scanner επιτρέπει στο πρόγραμμα να διαβάζει τι πληκτρολογεί ο χρήστης.

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
System.out.print("Δώσε όνομα: ");
String name = sc.nextLine();
System.out.println("Γεια σου " + name);
```

5.2 next, nextInt, nextLine

Ο Scanner μπορεί να διαβάσει διαφορετικά πράγματα ανάλογα με τη μέθοδο που καλούμε.

- next(): διαβάζει μία λέξη.
- nextInt(): διαβάζει ακέραιο αριθμό.
- nextDouble(): διαβάζει δεκαδικό αριθμό.
- nextLine(): διαβάζει ολόκληρη γραμμή.

5.3 Συχνό λάθος με nextInt και nextLine

Μετά από nextInt μένει το Enter στην είσοδο. Αν ακολουθεί nextLine, συχνά χρειάζεται ένα επιπλέον nextLine για να καθαρίσει η γραμμή.

```
int age = sc.nextInt();
sc.nextLine(); // καθαρίζει το Enter
String fullName = sc.nextLine();
```

6. Δομές επιλογής: if, else, switch

6.1 if και else

Με το if το πρόγραμμα αποφασίζει αν θα εκτελέσει ένα κομμάτι κώδικα με βάση μια συνθήκη. Με το else δίνουμε τι θα γίνει αν η συνθήκη είναι false.

```
if (grade >= 5) {
    System.out.println("Πέρασες");
} else {
    System.out.println("Απέτυχες");
}
```

6.2 Εμφωλευμένα if

Όταν έχουμε πολλές περιπτώσεις, μπορούμε να χρησιμοποιήσουμε if - else if - else. Είναι σημαντικό να χρησιμοποιούμε άγκιστρα για να είναι ξεκάθαρο ποιο else ανήκει σε ποιο if.

```
if (grade >= 8.5) {
    System.out.println("Άριστα");
} else if (grade >= 7.5) {
    System.out.println("Πολύ καλά");
} else if (grade >= 5) {
    System.out.println("Πέρασες");
} else {
    System.out.println("Απέτυχες");
}
```

6.3 switch

Το switch χρησιμοποιείται όταν θέλουμε να επιλέξουμε ανάμεσα σε πολλές συγκεκριμένες τιμές. Είναι πιο καθαρό από πολλά if όταν ελέγχουμε την ίδια μεταβλητή.

```
switch (department) {
    case "IT":
        System.out.println("Τμήμα Πληροφορικής");
        break;
    case "ECON":
        System.out.println("Τμήμα Οικονομικών");
        break;
    default:
        System.out.println("Άγνωστο τμήμα");
}
```

6.4 break και switch

Στην κλασική μορφή του switch, το break σταματά την εκτέλεση της περίπτωσης. Αν λείπει, η εκτέλεση μπορεί να συνεχίσει και στις επόμενες περιπτώσεις.

7. Δομές επανάληψης: while, do-while, for

7.1 while

Η while επαναλαμβάνει εντολές όσο η συνθήκη παραμένει true. Μπορεί να μη μπει καθόλου αν η συνθήκη είναι false από την αρχή.

```
int i = 1;
while (i <= 5) {
    System.out.println(i);
    i++;
}
```

7.2 do-while

Η do-while εκτελεί το σώμα του βρόχου τουλάχιστον μία φορά και μετά ελέγχει τη συνθήκη.

```
int choice;
do {
    System.out.println("1. Συνέχεια");
    System.out.println("0. Έξοδος");
    choice = sc.nextInt();
} while (choice != 0);
```

7.3 for

Η for χρησιμοποιείται συχνά όταν ξέρουμε πόσες φορές θέλουμε να επαναλάβουμε κάτι.

```
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

7.4 break και continue

Το break σταματά ολόκληρη την επανάληψη. Το continue παραλείπει το υπόλοιπο της τρέχουσας επανάληψης και συνεχίζει στην επόμενη.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
}
```

8. Μέθοδοι

8.1 Γιατί χρησιμοποιούμε μεθόδους

Οι μέθοδοι μας βοηθούν να σπάμε ένα πρόγραμμα σε μικρότερα κομμάτια. Έτσι ο κώδικας γίνεται πιο καθαρός, πιο εύκολος στον έλεγχο και πιο εύκολος στην επαναχρησιμοποίηση.

- Μια μέθοδος μπορεί να παίρνει παραμέτρους.
- Μια μέθοδος μπορεί να επιστρέφει αποτέλεσμα.
- Αν δεν επιστρέφει αποτέλεσμα, έχει τύπο επιστροφής void.

8.2 Παράδειγμα μεθόδου

Η υπογραφή μιας μεθόδου περιλαμβάνει το όνομα, τις παραμέτρους και τον τύπο επιστροφής.

```
public static int add(int a, int b) {
    return a + b;
}
int result = add(3, 4);
```

8.3 Μέθοδοι αντικειμένων

Σε αντικειμενοστρεφή κώδικα, πολλές μέθοδοι ανήκουν σε αντικείμενα και χρησιμοποιούν τα πεδία του αντικειμένου.

```
class Counter {
    private int value;
    public void increase() {
        value++;
    }
    public int getValue() {
        return value;
    }
}
```

9. Κλάσεις, αντικείμενα, constructors και this

9.1 Πεδία και μέθοδοι

Μια κλάση έχει πεδία για τα δεδομένα και μεθόδους για τη συμπεριφορά. Τα πεδία καλό είναι να είναι private και η πρόσβαση να γίνεται με μεθόδους.

```
class Student {
    private String name;
    private int semester;
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

9.2 Constructor

Ο constructor εκτελείται όταν δημιουργείται νέο αντικείμενο. Τον χρησιμοποιούμε για να δώσουμε αρχικές τιμές.

```
class Student {
    private String name;
    public Student(String name) {
        this.name = name;
    }
}
```

9.3 this

Το this αναφέρεται στο τρέχον αντικείμενο. Είναι πολύ χρήσιμο όταν μια παράμετρος έχει ίδιο όνομα με ένα πεδίο.

```
public Student(String name) {
    this.name = name;
}
```

9.4 Encapsulation

Η ενθυλάκωση σημαίνει ότι κρύβουμε τα εσωτερικά δεδομένα μιας κλάσης και επιτρέπουμε πρόσβαση μόνο μέσα από ελεγχόμενες μεθόδους.

- Τα πεδία συνήθως δηλώνονται private.
- Χρησιμοποιούμε getters και setters όπου χρειάζεται.
- Η κλάση προστατεύει τα δεδομένα της από λάθος χρήση.

10. Τύποι αναφοράς και ισότητα αντικειμένων

10.1 Τι κρατάει μια μεταβλητή αντικειμένου

Μια μεταβλητή τύπου κλάσης δεν κρατά το ίδιο το αντικείμενο. Κρατά μια αναφορά προς το αντικείμενο στη μνήμη.

```
Student s1 = new Student("Maria");
Student s2 = s1;
// s1 και s2 δείχνουν στο ίδιο αντικείμενο
```

10.2 null

Η τιμή null σημαίνει ότι η μεταβλητή δεν δείχνει σε κανένα αντικείμενο. Αν προσπαθήσουμε να καλέσουμε μέθοδο σε null, θα έχουμε σφάλμα εκτέλεσης.

```
Student s = null;
// s.getName(); // λάθος: NullPointerException
```

10.3 == και equals

Το == σε αντικείμενα ελέγχει αν δύο μεταβλητές δείχνουν στο ίδιο αντικείμενο. Η equals χρησιμοποιείται για σύγκριση περιεχομένου, εφόσον έχει υλοποιηθεί σωστά.

```
public boolean equals(Object o) {
    if (o instanceof Student) {
        Student other = (Student) o;
        return this.name.equals(other.name);
    }
    return false;
}
```

10.4 instanceof και casting

Το instanceof ελέγχει αν ένα αντικείμενο είναι συγκεκριμένου τύπου. Μετά από αυτό μπορούμε να κάνουμε ασφαλέστερα casting.

```
if (o instanceof Student) {
    Student s = (Student) o;
    System.out.println(s.getName());
}
```

11. Πίνακες και ArrayList

11.1 Πίνακες

Ο πίνακας αποθηκεύει πολλές τιμές ίδιου τύπου. Τα στοιχεία αριθμούνται από το 0 μέχρι length - 1.

```
int[] grades = new int[5];
grades[0] = 8;
grades[1] = 7;
System.out.println(grades.length);
```

11.2 Διάσχιση πίνακα

Συνήθως χρησιμοποιούμε for για να περάσουμε από όλα τα στοιχεία ενός πίνακα.

```
for (int i = 0; i < grades.length; i++) {
    System.out.println(grades[i]);
}
```

11.3 Πίνακες αντικειμένων

Ένας πίνακας μπορεί να περιέχει αναφορές σε αντικείμενα. Πρέπει όμως να δημιουργήσουμε και τα αντικείμενα, όχι μόνο τον πίνακα.

```
Student[] students = new Student[3];
students[0] = new Student("Maria");
students[1] = new Student("Nikos");
```

11.4 Δισδιάστατοι πίνακες

Οι δισδιάστατοι πίνακες χρησιμοποιούνται για δεδομένα τύπου πίνακα/μήτρας, δηλαδή γραμμές και στήλες.

```
int[][] matrix = new int[3][4];
matrix[0][0] = 10;
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

11.5 ArrayList

Η ArrayList μοιάζει με πίνακα αλλά μπορεί να μεγαλώνει δυναμικά. Είναι χρήσιμη όταν δεν ξέρουμε από πριν πόσα στοιχεία θα έχουμε.

```
import java.util.ArrayList;
ArrayList<String> names = new ArrayList<>();
names.add("Maria");
names.add("Nikos");
System.out.println(names.get(0));
System.out.println(names.size());
```

12. Κληρονομικότητα, super και overriding

12.1 Κληρονομικότητα

Η κληρονομικότητα επιτρέπει σε μια κλάση να πάρει χαρακτηριστικά και μεθόδους από μια άλλη κλάση. Η γενική κλάση λέγεται γονική και η ειδική υποκλάση.

```
class User {
    private String username;
    public String getUsername() {
        return username;
    }
}

class AdminUser extends User {
    public void deleteUser(User u) {
        System.out.println("Deleting user");
    }
}
```

12.2 super

Με τη λέξη super καλούμε constructor ή μέθοδο της γονικής κλάσης.

```
class AdminUser extends User {
    public AdminUser(String username) {
        super(username);
    }
}
```

12.3 Method overriding

Overriding έχουμε όταν μια υποκλάση ξαναγράφει μέθοδο που υπάρχει στη γονική κλάση. Έτσι δίνει πιο ειδική συμπεριφορά.

```
class User {
    public void login() {
        System.out.println("User login");
    }
}

class AdminUser extends User {
    public void login() {
        super.login();
        System.out.println("Admin privileges enabled");
    }
}
```

12.4 toString

Η toString επιστρέφει κείμενο που περιγράφει ένα αντικείμενο. Είναι καλή πρακτική να την υλοποιούμε σε κλάσεις που θέλουμε να εμφανίζονται καθαρά.

```
public String toString() {  
    return "User: " + username;  
}
```

12.5 Overloading vs overriding

Overloading σημαίνει ίδιο όνομα μεθόδου αλλά διαφορετικές παράμετροι. Overriding σημαίνει ίδια μέθοδος σε υποκλάση με νέα υλοποίηση.

13. Πολυμορφισμός, αφηρημένες κλάσεις και διεπαφές

13.1 Πολυμορφισμός

Πολυμορφισμός σημαίνει ότι μπορούμε να χειριζόμαστε διαφορετικά αντικείμενα μέσα από έναν κοινό τύπο. Έτσι γράφουμε γενικό κώδικα που δουλεύει με πολλές ειδικές κλάσεις.

```
User u = new AdminUser("root");  
u.login(); // εκτελείται η login του πραγματικού αντικειμένου
```

13.2 Αφηρημένη κλάση

Μια αφηρημένη κλάση είναι πολύ γενική για να δημιουργούμε αντικείμενα από αυτή. Χρησιμοποιείται για να δώσει κοινή δομή στις υποκλάσεις.

```
abstract class DeliveryVehicle {  
    private String id;  
    public abstract void deliverPackage(Box b);  
}
```

13.3 Αφηρημένες μέθοδοι

Μια αφηρημένη μέθοδος δηλώνει τι πρέπει να υπάρχει, αλλά όχι πώς θα γίνει. Οι υποκλάσεις υποχρεώνονται να δώσουν υλοποίηση.

```
class Truck extends DeliveryVehicle {  
    public void deliverPackage(Box b) {  
        System.out.println("Delivery by truck");  
    }  
}
```

13.4 Διεπαφή

Η διεπαφή είναι συμβόλαιο. Περιγράφει ποιες μεθόδους πρέπει να έχει μια κλάση. Δεν μας ενδιαφέρει η εσωτερική υλοποίηση, αλλά η κοινή συμπεριφορά.

```
interface Deliverable {  
    void deliverPackage(Box b);  
    void prepareForDelivery();  
}
```

13.5 implements

Μια κλάση υλοποιεί μια διεπαφή με τη λέξη implements και πρέπει να δώσει σώμα σε όλες τις μεθόδους της διεπαφής.

```
class Bike implements Deliverable {  
    public void deliverPackage(Box b) {  
        System.out.println("Delivery by bike");  
    }  
    public void prepareForDelivery() {  
        System.out.println("Prepare bike");  
    }  
}
```

13.6 Πότε abstract class και πότε interface

Χρησιμοποιούμε abstract class όταν υπάρχει κοινή βάση, κοινά πεδία ή κοινή μερική υλοποίηση. Χρησιμοποιούμε interface όταν θέλουμε να δηλώσουμε κοινή ικανότητα/συμπεριφορά που μπορεί να έχουν διαφορετικές κλάσεις.

14. Είσοδος/Έξοδος και αρχεία

14.1 Τι είναι είσοδος και έξοδος

Είσοδος είναι τα δεδομένα που παίρνει το πρόγραμμα. Έξοδος είναι τα δεδομένα που εμφανίζει ή αποθηκεύει. Στα αρχεία, το πρόγραμμα μπορεί να διαβάζει αποθηκευμένα δεδομένα και να γράφει νέα.

14.2 Η κλάση File

Η File αναπαριστά μια διαδρομή προς αρχείο ή φάκελο. Από μόνη της δεν διαβάζει ούτε γράφει περιεχόμενο, απλώς περιγράφει το αρχείο.

```
File f = new File("input.txt");
System.out.println(f.getName());
System.out.println(f.getAbsolutePath());
System.out.println(f.exists());
```

14.3 Ανάγνωση αρχείου με Scanner

Μπορούμε να χρησιμοποιήσουμε Scanner για να διαβάσουμε γραμμή-γραμμή ή λέξη-λέξη από αρχείο.

```
import java.io.File;
import java.util.Scanner;

Scanner sc = new Scanner(new File("input.txt"));
while (sc.hasNextLine()) {
    String line = sc.nextLine();
    System.out.println(line);
}
sc.close();
```

Τελική ιδέα για τους πρωτοετείς: η Java δεν είναι μόνο σύνταξη. Είναι τρόπος οργάνωσης σκέψης. Ξεκινάμε με μεταβλητές και if, συνεχίζουμε με μεθόδους, και μετά οργανώνουμε τον κώδικα σε κλάσεις και αντικείμενα.